

Отчёт по домашней работе 5

Реализация межсервисного взаимодействия посредством очередей сообщений

Милютин Никита, БР1.2

1. Тема

Реализация асинхронного межсервисного взаимодействия в микросервисной архитектуре с использованием RabbitMQ.

2. Цель работы

Подключить и настроить брокер сообщений RabbitMQ, а также реализовать обмен событиями между микросервисами приложения бронирования столиков в ресторанах.

3. Выбранный брокер сообщений

Для работы выбран RabbitMQ. Он подходит для событийной коммуникации между сервисами, поддерживает exchange, routing key, очереди, подтверждение обработки сообщений и management-интерфейс.

RabbitMQ запускается через `docker-compose.yml`:

```
image: rabbitmq:3.13-management
ports:
  - "5672:5672"
  - "15672:15672"
```

Порт 5672 используется приложениями для AMQP-подключения, порт 15672 используется для веб-интерфейса RabbitMQ.

4. Реализованные сервисы

Сервис	Порт	Назначение
Reservation Service	4101	Создаёт и отменяет бронирования, публикует события
Notification Service	4102	Подписывается на события бронирований и создаёт уведомления
RabbitMQ	5672, 15672	Брокер сообщений и management UI

5. Схема взаимодействия

Reservation Service не вызывает Notification Service напрямую. Вместо этого он публикует доменные события в RabbitMQ exchange restaurant-booking.events.

Notification Service создаёт очередь notification-service.reservation-events, привязывает её к exchange по routing key reservation.* и обрабатывает все события бронирований.

1. Пользователь создаёт бронирование через Reservation Service.
2. Reservation Service сохраняет бронирование в своём хранилище.
3. Reservation Service публикует событие reservation.created.
4. RabbitMQ доставляет сообщение в очередь Notification Service.
5. Notification Service получает сообщение, подтверждает обработку и создаёт уведомление.

6. Контракты сообщений

Используется topic exchange:

- exchange: restaurant-booking.events;
- routing keys: reservation.created, reservation.cancelled;
- queue: notification-service.reservation-events;
- binding key: reservation.*.

Пример события создания бронирования:

```
{
  "event_id": "evt_1710000000000",
  "event_type": "reservation.created",
  "occurred_at": "2026-04-05T12:00:00.000Z",
  "payload": {
    "reservation_id": 2,
    "user_id": 1,
    "restaurant_id": 1,
    "table_id": 2,
    "reservation_datetime": "2026-04-06T19:00:00.000Z",
    "guest_count": 4,
    "status": "confirmed"
  }
}
```

Пример события отмены бронирования:

```
{
  "event_id": "evt_1710000000001",
  "event_type": "reservation.cancelled",
  "occurred_at": "2026-04-05T12:05:00.000Z",
  "payload": {
    "reservation_id": 2,
    "user_id": 1,
    "status": "cancelled"
  }
}
```

```
}  
}
```

7. Надёжность обработки

В реализации используются:

- durable exchange;
- durable queue;
- persistent messages через `deliveryMode: 2`;
- ручное подтверждение обработки через `channel.ack`;
- отдельный consumer, не связанный напрямую с producer.

Такой подход снижает связность сервисов: Reservation Service может публиковать события, не зная о количестве подписчиков.

8. Запуск и проверка

Запуск RabbitMQ:

```
docker compose up -d
```

Запуск Notification Service:

```
npm run dev:notification
```

Запуск Reservation Service:

```
npm run dev:reservation
```

Создание бронирования:

```
curl -X POST http://localhost:4101/reservations \
  -H "Content-Type: application/json" \
  -d '{"user_id":1,"restaurant_id":1,"table_id":2,
"reservation_datetime":"2026-04-06T19:00:00.000Z","guest_count":4}'
```

Проверка уведомлений:

```
curl http://localhost:4102/notifications
```

9. Результат

В результате домашней работы подключён RabbitMQ и реализовано асинхронное межсервисное взаимодействие. Reservation Service публикует события о создании и отмене бронирования, а Notification Service получает эти события через очередь и создаёт уведомления.